

4장 기계어 : 만들어진 시스템을 분석하면서 어떻게 설계했는지 의도를 파악해라.

*기계어

-기계 수준의 명령어들로 이루어진 저수준 프로그램을 코딩할 수 있게 미리 정의된 규칙이다.

ex: 프로세서의 산술 및 논리연산/ 메모리에서 값 불러오기,적재/ 레지스터간 이동

*기계어는 하드웨어와 소프트웨어가 만나는 접점이자 추상적 기호이다.

명령어는 실리콘 소자내에서 실행되는 물리적 연산으로 바뀐다.

대부분의 사람들은 기계어로 직접 프로그램을 작성할 일이 없지만, 컴퓨터의 아키텍처를 완전히 이해하려면 저수준 프로그래밍을 공부해야한다.

*메모리 : 데이터와 메모리 저장하는 하드웨어 장치의 통칭

*프로세서 : CPU

*레지스터 : 메모리 접근 연산은 상대적으로 느리며, 긴 명령어가 필요하다.

주소 하나 표현하는데 32비트가 필요하므로

대부분의 프로세서는 값 하나 저장하는 레지스터 여러개 두고 있다.

이 레지스터는 프로세서 바로 옆에 위치한다. >> 로컬 고속메모리 역할한다.

*2진 코드는 암호같아 보이므로 기계어는 2진코드와 연상 기호를 같이 사용한다.(ADD)

*R1 R3 ADD >> 텍스트 처리프로그램 >> 2진표현 순으로 진행한다.

이러한 이진 기호 표기법을 어셈블리어

*어셈블리어를 2진 코드로 번역하는 프로그램을 어셈블러 (어셈블러는 기계어의 한 종류)

*명령어

1.산술 및 논리연산.

```
ADD R2,R1,R3 // R2←R1+R3 where R1,R2,R3 are registers
ADD R2,R1,foo // R2←R1+foo where foo stands for the
               // value of the memory location pointed
               // at by the user-defined label foo.
AND R1,R1,R2 // R1←bit wise And of R1 and R2
```

2.메모리 접근

1). 직접주소지정방식

:특정주소 쓰거나, 그 주소 나타내는 기호 활용

```
LOAD R1,67 // R1←Memory[67]
// Or, assuming that bar refers to memory address 67:
LOAD R1,bar // R1←Memory[67]
```

2). 즉시 주소지정방식

:명령어 코드에 있는 상수 불러오는데 쓰인다.

```
LOADI R1,67 // R1←67
```

3). 간접주소지정

:필요한 주소값을 저장하고 있는 메모리 위치를 참조하는 명령어 사용. 포인터이다.

```
// Translation of x=foo[j] or x={foo+j}):
ADD R1,foo,j // R1←foo+j
LOAD* R2,R1 // R2←Memory[R1]
STR R2,x // x←R2
```

*제어 흐름

분기->반복/조건실행/서브루틴호출

무조건 점프 : 가고자하는 위치의 주소만 명시

조건 점프 : 조건 기입

*Hack Computer

-폰 노이만구조

-CPU/ 명령용 메모리/ 데이터용 메모리/ 2개의 메모리 맵핑 IO장치

-16비트 장치

*메모리 주소공간

1.명령어 메모리

2.데이터 메모리(16비트 폭에 주소 15비트이다)

핵의 CPU는 명령어 메모리에 존재하는 프로그램만 수행

*레지스터

16비트 레지스터 D와 A

D는 데이터값 저장 용도로 씀

A는 데이터 레지스터와 주소 어드레스 2가지 역할 가능

주소 접근은 M으로, M은 항상 A레지스터의 현재값을 주소로 하는 메모리 내 단어 참조한다.

M으로 해야 비트 많이 안 먹고 16비트 내에서 처리가능 하니까.

(책갈피처럼 빠르게 업무 처리하는 보조용 레지스터인것인가?)

M은 항상 A레지스터의 현재 값을 주소로 하는 메모리 내 데이터를 참조한다.

EX) D=Memory[516]-1

-1). A에 516 넣고

-2). D=M-1 명령어 실행

A 레지스터는 명령어 메모리에 직접 접근하는 용도로 쓴다.

핵의 점프 명령어에서는 주소를 직접 쓰지 않는다.

무조건 A의 현재값을 주소로 하는 명령어로 점프한다.

(A가 517 가지고 있으면 517 주소를 가진 명령어 씀)

goto35하고 싶으면 A에 35 넣고 goto한다. (주소없이 goto 명령)

@value는 A레지스터에 value가 참조하는 값을 넣으라는 것

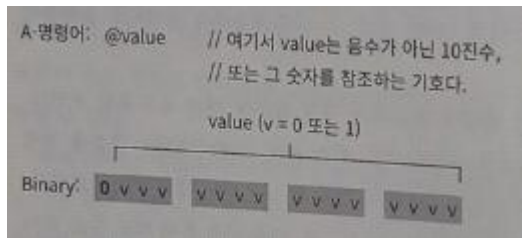
*명령어

-A명령어 (주소명령어)

-C명령어 (계산명령어)

1.A 명령어

:A 레지스터에 15비트 값을 설정하는데 쓰인다.



@value라는 명령어는 0xxx_xxxx_xxxx_xxxx 라는 이진코드 가진다.

@5는 0000_0000_0000_0101이다.

A 명령어는 3가지 용도로 쓰인다.

- 1. 상수를 입력하는 용도
- 2. 다음 C 명령어에서 해당 주소에 있는 값을 참조할 수 있도록
A 레지스터에 데이터의 주소를 미리 입력해 두는 용도
- 3. A 레지스터에 점프할 주소를 미리 입력해서, 그 다음
점프 C 명령어 준비하는 용도이다.

C language

```
// Adds 1+...+100.  
int i = 1;  
int sum = 0;  
While (i <= 100){  
    sum += i;  
    i++;  
}
```

Hack machine language

```
// Adds 1+...+100.  
#i // i refers to some mem. location.  
M=1 // i=1  
#sum // sum refers to some mem. location.  
M=0 // sum=0  
(LOOP)  
#i  
D=M // D=i  
#100  
D=D-A // D=i-100  
#END  
D;JGT // If (i-100)>0 goto END  
#i  
D=M // D=i  
#sum  
M=D+M // sum=sum+i  
#i  
M=M+1 // i=i+1  
#LOOP  
J;JMP // Goto LOOP  
(END)  
#END  
J;JMP // Infinite loop
```

@는 어떤 남은 레지스터 주소 다루겠다는 소리

(A레지스터에 그 주소를 넣는다)

M은 A레지스터에 있는 주소를 다룬다는 이야기의 축약형

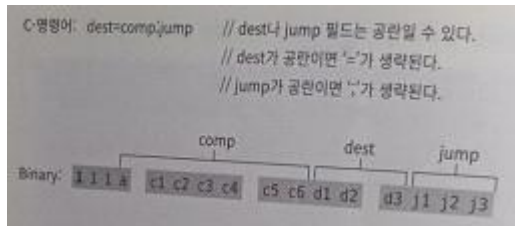
2. C 명령어

C 명령어는 핵 플랫폼에서 거의 모든 일 수행한다.

3가지 방향을 가지고 있다

- 1. 무엇을 계산하는지
- 2. 계산된 값을 어디에 저장하는지?
- 3. 그 다음에 할 일은 무엇인지

[C 명령어의 이진코드]



MSB 값 1은 이 명령어가 C명령어임을 나타내는 것이다.

그 다음 2비트는 사용하지 않는다.

- comp : ALU가 할 연산이 무엇인지를 뜻한다.
- dest : 계산된 값을 어디에 저장할지를 뜻한다.
- jump : 점프 조건으로 다음에 불러와서 실행할 명령어가 무엇인지를 의미한다.

1). Comp 필드 (Composition)

M은 Memory[A]를 뜻한다.

구성: a비트와 c1..c6 비트로 구성

(a=0일 때) comp 연산기호	c1	c2	c3	c4	c5	c6	(a=1일 때) comp 연산기호
0	1	0	1	0	1	0	
1	1	1	1	1	1	1	
-1	1	1	1	0	1	0	
D	0	0	1	1	0	0	
A	1	1	0	0	0	0	M
!D	0	0	1	1	0	1	
!A	1	1	0	0	0	1	!M
-D	0	0	1	1	1	1	
-A	1	1	0	0	1	1	-M
D+1	0	1	1	1	1	1	
A+1	1	1	0	1	1	1	M+1
D-1	0	0	1	1	1	0	
A-1	1	1	0	0	1	0	M-1
D+A	0	0	0	0	1	0	D+M
D-A	0	1	0	0	1	1	D-M
A-D	0	0	0	1	1	1	M-D
D&A	0	0	0	0	0	0	D&M
D/A	0	1	0	1	0	1	D/M

2). Dest 필드 (Destination)

d1	d2	d3	연상기호	목적지(계산된 값을 저장할 위치)
0	0	0	null	값은 어디에도 저장되지 않음
0	0	1	M	Memory[A](A 값을 주소로 하는 메모리 레지스터)
0	1	0	D	D레지스터
0	1	1	MD	Memory[A]와 D레지스터
1	0	0	A	A레지스터
1	0	1	AM	A레지스터와 Memory[A]
1	1	0	AD	A레지스터와 D레지스터
1	1	1	AMD	A레지스터, Memory[A], D레지스터

표 4.4 C-명령어의 dest 필드

3). Jump 필드

컴퓨터가 다음에 수행할 일을 지시한다.

- 다음번 명령을 불러와서 실행하는 것
- 프로그램 내 다른 곳에서 위치한 명령을 불러와서 실행하는 일 (A레지스터에 점프할 주소가 있다는 가정)

j1 (out < 0)	j2 (out = 0)	j3 (out > 0)	연상기호	효과
0	0	0	null	점프하지 않음
0	0	1	JGT	out > 0이면 점프
0	1	0	JEQ	out = 0이면 점프
0	1	1	JGE	out ≥ 0이면 점프
1	0	0	JLT	out < 0이면 점프
1	0	1	JNE	out ≠ 0이면 점프
1	1	0	JLE	out ≤ 0이면 점프
1	1	1	JMP	점프

(마지막 JMP는 무조건 점프이다)

//JEQ : if(comp jump 0) jump to execute the instruction in ROM[A]

3. 기호

어셈블리 명령은 상수나 기호를 이용해서 메모리 주소를 참조할 수 있다.

기호활용은 3가지 방식이 있다.

-미리 정의된 기호

>가상 레지스터(R0-R15는 RAM의 주소 0-15 가르킨다)

>미리 정의된 포인터 (SP, LCL, ARG, THIS, THAT 은 RAM 0-4를 가르킨다)

>IO 포인터 : (SCREEN과 KBD는 각각 RAM 주소 16384, 24576 참조한다.)

: 스크린 및 키보드의 메모리 매핑 시작 주소가 된다.

-레이블 기호

goto 명령어의 목적지를 나타내는 것.

사용자가 (Xxx)라는 의사명령으로 직접 선언한다.

다음에 실행할 명령을 담고 있는 명령어 메모리 주소를 기호 Xxx로 선언하라는 뜻.

레이블은 한번 선언시 어셈블리 프로그램 내 어디서든 쓸 수 있다.

(심지어 선언된 라인의 앞쪽 라인에서도 쓸 수 있다.)

-변수 기호

Xxx나 미리 정의된 기호가 아닌 모든 사용자 정의 기호는 변수 취급한다.

RAM 주소 16에서부터 차례대로 변수마다 유일한 메모리 주소를 할당한다.

4.입력/출력 조작

-스크린과 키보드는 메모리 맵을 통해 컴퓨터 플랫폼과 통신한다.

(메모리 세그먼트에 2진값을 쓰면 그에 대응하는 스크린 위에 픽셀이 그려짐)

(각 픽셀에 0 또는 1을 통해 출력한다고 보자)

-키보드 입력은 해당 키에 대응하는 메모리 위치를 읽어 들이는 식으로 처리.

1)스크린

512열 X 256 행의 흑백 픽셀로 구성

픽셀은 RAM 16384부터 시작하고 8K짜리 메모리 맵에 대응한다.

각 행은 화면 왼쪽 상단부터 시작.

RAM내에서 32개의 16비트 워드로 표현.

위에서 r번째행, 왼쪽에서 c번째 열의 픽셀은 $RAM[16384+r*32+ c/16]$

$32 \times 16 = 512$ 비트 이므로 각각의 비트가 512개 열의 픽셀에 대응한다.

2)키보드

RAM 주소 24576에 위치한 1워드 메모리맵을 통해 물리적 키보드 장치와 통신한다.

키가 눌릴 때마다. 눌린 키의 16비트 ASCII코드가 RAM[24576]에 기록되는 식.

키가 안눌리면 그 메모리 위치에 코드 0이 기록된다.

눌린 키	코드	눌린 키	코드
새 라인	128	엔드(end)	135
백스페이스	129	페이지 업(page up)	136
왼쪽 화살표	130	페이지 다운(page down)	137
위쪽 화살표	131	인서트(insert)	138
오른쪽 화살표	132	딜리트(delete)	139
아래쪽 화살표	133	이스케이프(esc)	140
홈(home)	134	f1~f12	141~152

그림 4.6 맥 컴퓨터용 키보드 코드

4. 구문규칙과 파일 형식

*2진 코드 파일

-2진코드파일은 텍스트 라인들로 구성.

-각 라인은 16개의 ASCII문자 0,1로 구성.

-하나의 기계어 명령어를 부호화한 것이다.

-파일내 모든 라인들이 모여 하나의 기계어 프로그램을 이룬다.

-명령어 메모리에 기계어 프로그램이 로드될때는 파일의 n번째 라인에 있는 2진코드가 명령어 메모리의 주소 n에 저장되는 규칙을 따른다.

-기계어 프로그램은 'hack'확장자인 텍스트 파일에 저장.

*어셈블리 언어 파일

-'asm'확장자를 가지는 텍스트 파일

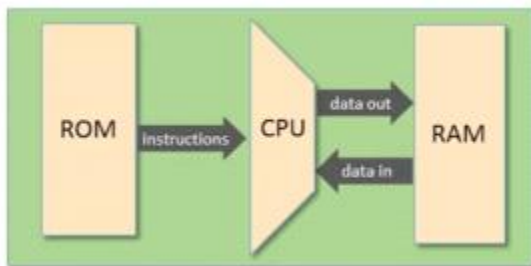
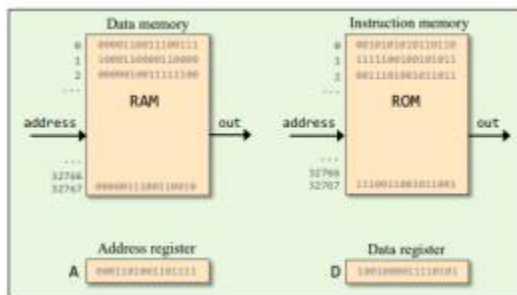
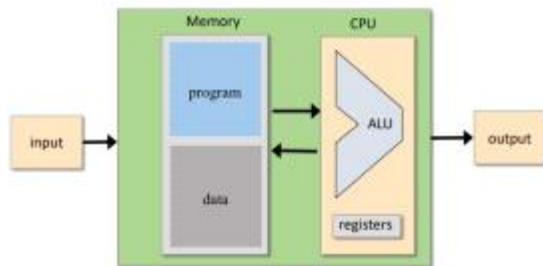
-텍스트 라인으로 구성되며 각 텍스트 라인은 명령어/기호선언을 뜻한다.

-*기호 선언은 다음번 명령이 저장되는 메모리 위치에 심볼 레이블을 할당하라고 어셈블러에게 지시한다.

<정리>

1.핵 컴퓨터 스펙

- 명령어 메모리 모듈 (16 비트 폭/ 주소비트 15비트) (ROM역할)
(핵의 CPU는 명령어 메모리에 존재하는 프로그램만 수행한다.)
- 데이터 메모리 모듈 (16 비트 폭/ 주소비트 15비트) (RAM역할)
- 스크린/키보드 (16비트 장치)
- CPU
- 폰 노이만 구조



2.레지스터

- 1). D 레지스터 : 데이터 저장하는 역할로 16비트 레지스터 1개이다.
 - 2). A 레지스터 : >>데이터 메모리에 직접 접근을 용이하게 하는 용도로 쓰임
>>명령어 메모리에 직접 접근을 하는 용도로 쓰임
>>16비트 레지스터 1개이다.
- (레지스터는 메모리 로드 시간이 오래 걸리므로 바로 옆에 둔 저장장치이다.)

3.명령어

명령어 1개는 16비트 폭이다.

데이터 또한 16비트 폭이므로 주소와 데이터를 한 번에 명령어에 넣을 수 없다.

* M 명령어

이를 해결하기 위해 M이라는 이름의 주소로만 메모리 접근이 가능하도록 강제한다.

M=Memory[A] 라고 보면 된다.

*명령어는 2가지로 나눌 수 있다.

1). A-명령어

A 레지스터에 15 비트값 설정하는 데 쓰인다. (레지스터에 값 저장용)

@value 명령어도 A-명령어이다.

@value라는 명령어는 A 레지스터에 0xxx_xxxx_xxxx_xxxx 저장한다.

ex)@5는 0000_0000_0000_0101을 A 레지스터에 저장한다.

* @value 명령어

value라는 값을 A 레지스터에 저장하라는 것

ex) @100 // A 레지스터에 100넣음 >> RAM[100]에 접근한다.

ex) @i // i라는 변수 주소 A 레지스터에 넣는다.

ex)

// Set RAM[100] to -1

@100 // A=100

M=-1 // RAM[100]=-1

ex)

// Set RAM[300] to the value of the D register minus 1

@300

M=D-1

2). C-명령어

연산을 수행하는 명령어

C-명령어는 dest=comp;jmp 구조이다.(저장장소/연산식/점프식)

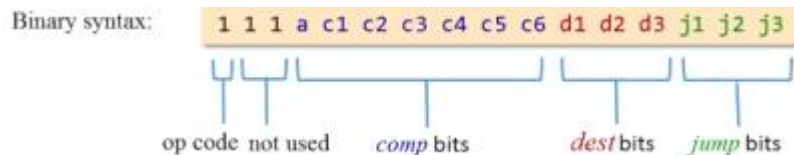
C-명령어의 이진코드 = 111a_cccc_ccdd_djjj 형태이다.

(?? C-명령어의 이진코드는 어디에 저장하는가?)

>> 명령어 메모리에 저장한다. 텍스트 파일에 있는 기계어 프로그램을 명령어 메모리로 로드하는 기능 제공.

ex)

```
// IF (D-1==0) jump to execute the instruction stored in ROM[56]
@56
D-1;JEQ //if (D-1==0) goto 56
//D-1 연산이 잘 되었는지 여부가 out이다
//dest=comp;jump ; out is a result of comp.
//In that, JEQ(if(out==0) jump) works
//@56 sets A=56 then, JEQ access ROM[A]
//JEQ : if(comp jump 0) jump to execute the instruction in ROM[A]
//goto는 ROM[A]로 접속한다.
```



4. 스크린

-스크린은 행 256 / 열 512개의 픽셀로 구성된다.

-스크린의 픽셀값을 저장하는 위치는 RAM의 16384번 레지스터부터 시작이다.

(RAM의 레지스터는 16비트인 워드 1개이다.)

-r번째 행의 c열의 픽셀은 RAM[16384+r*32+c/16]에 위치한 레지스터의 c%16번째 비트와 대응한다.

pf) r행의 c열에 위치한 픽셀은 r*512+c번째 픽셀이다.

RAM[16384+(r*512+c)/16]에 위치한 레지스터에 대응하는 비트가 있을 것이다.

열은 (r*512+c)%16 이다. {(r*512+c)%16}=c%16이다.

그래서 위의 식이 나오는 것이다.

5.키보드

Key	Code	Key	Code	Key	Code	Key	Code
()	48	(space)	32	:	58	newline	128
1	49	!	33	;	59	backspace	129
-	50	"	34	<	60	left arrow	130
9	57	#	35	=	61	up arrow	131
		\$	36	>	62	right arrow	132
		%	37	?	63	down arrow	133
		&	38	@	64	home	134
		'	39			end	135
A	65	(	40			Page up	136
B	66	)	41	[	91	Page down	137
...	...	*	42	/	92	insert	138
Z	90	+	43	]	93	delete	139
		,	44	^	94	esc	140
		.	45	_	95	f1	141
		/	46			...	...
		/	47			f12	152

When no key is pressed, the resulting code is 0

6.built in symbol

The Hack assembly language features *built-in symbols*:

symbol	value
R0	0
R1	1
R2	2
...	...
R15	15

These symbols can be used to denote "virtual registers":

instead of:

```
// RAX[5]=15
#15
D=A

#5
M=D
```

better style:

```
// RAX[5]=15
#15
D=A

#R5
M=D
```

symbol	value
R0	0
R1	1
R2	2
...	...
R15	15
SCREEN	16384
KBD	24576

symbol	value
SP	0
LCL	1
ARG	2
THIS	3
THAT	4

7. Programming

```

// Program: Signum.asm
// Computes: if R0>0
//           R1=1
//           else
//           R1=0

0  @R0
1  D=M      // D = RAM[0]
2
3  @R0
4  D;JGT    // If R0>0 goto 8
5
6  @R1
7  M=0      // RAM[1]=0
8
9  @R1
10 M=1      // R1=1
11
12 @R1
13 JMP

```

```

// Program: Signum.asm
// Computes: if R0>0
//           R1=1
//           else
//           R1=0

0  @R0
1  D=M
2
3  @POSITIVE
4  D;JGT
5
6  @R1
7  M=0
8  @END
9  JMP
10
11 (POSITIVE)
12 @R1
13 M=1
14
15 (END)
16 @END
17 JMP

```

```

// Program: Flip.asm
// Flips the values of
// RAM[0] and RAM[1]
// temp = R1
// R1 = R0
// R0 = temp

0  @R1
1  D=M
2  @temp
3  M=D      // temp = R1
4
5  @R0
6  D=M
7  @R1
8  M=D      // R1 = R0
9
10 @temp
11 D=M
12 @R0
13 M=D      // R0 = temp
14
15 (END)
16 @END
17 JMP

```

Iterative processing example: compute $1 + 2 + \dots + n$

Hack assembly code

```

// Program: Sum1toN.asm
// Computes RAM[1] = 1+2+...+n
// Usage: put a number (n) in RAM[0]

0  @R0
1  D=M
2  @n
3  M=D      // n = R0
4
5  @i
6  M=1      // i = 1
7
8  @sum
9  M=0      // sum = 0
10
11 ...

```

Pseudo code

```

// Computes RAM[1] = 1+2+...+RAM[0]

n = R0
i = 1
sum = 0
LOOP:
  if i > n goto STOP
  sum = sum + i
  i = i + 1
  goto LOOP
STOP:
  R1 = sum

```

(continued)

```

12 (LOOP)
13 @i
14 D=M
15 @n
16 D-D-M
17 @STOP
18 D;JGT    // if i > n goto STOP
19
20 @sum
21 D=M
22 @i
23 D-D-M
24 @sum
25 M=D      // sum = sum + i
26
27 @i
28 M=M+1    // i = i + 1
29 @LOOP
30 JMP
31
32 (STOP)
33 @sum
34 D=M
35 @R1
36 M=D      // RAM[1] = sum
37
38 (END)
39 @END
40 JMP

```

Labels

Example (pseudocode)

```

1 = 1000
LOOP:
  if (1 = 0) goto CONT
  1 = 1 - 1
  goto LOOP
CONT:
  ...

```

write

Hack assembly

```

1 1 = 1000
2 @1000
3 D=A
4 @1
5 M=D
6 (LOOP)
7 if (1 = 0) goto CONT
8 @1
9 D=M
10 @CONT
11 D;JEQ
12 1 = 1 - 1
13 @1
14 M=M-1
15 goto LOOP
16 @LOOP
17 JMP
18 (CONT)
19 ...

```

Label declaration in the Hack assembly language:

{sym}

Results in binding sym to the address of the next instruction

In this example:

LOOP is bound to 4

CONT is bound to 12

(done by the assembler; The code writer doesn't care about these numbers)

@LOOP, @CONT는 라벨같은것

그냥 값은 의미가 없고 이름이 의미있는것

j1 (out < 0)	j2 (out = 0)	j3 (out > 0)	연상기호	효과
0	0	0	null	점프하지 않음
0	0	1	JGT	out > 0이면 점프
0	1	0	JEQ	out = 0이면 점프
0	1	1	JGE	out ≥ 0이면 점프
1	0	0	JLT	out < 0이면 점프
1	0	1	JNE	out ≠ 0이면 점프
1	1	0	JLE	out ≤ 0이면 점프
1	1	1	JMP	점프

*Max.asm

@R0

D=M

@R1

D=D-M

@POS

D;JGT

@R1

D=M

@END

0;JMP

(POS)

@R0

D=M

(END)

@R1

D=M

(INFINITE_LOOP)

@INFINITE_LOOP

0;JMP // infinite loop

@는 변수로 쓰거나 라벨(강 표시 정도)로 쓸 수 있다.